11

PART 3: PRACTICE AND IMPLICATIONS

The specification economy

What changes in software economics once specifications, not code, are the asset that persists?

At the retail bank from Chapter 9, the ledger's second migration arrives sooner than anyone expected. Not away from COBOL this time. COBOL is long gone, replaced years earlier by the Java service the recovery project built. Java is now the system under review. A newer platform the bank's infrastructure team has been trialing runs the nightly reconciliation job at a fraction of the cost, and finance wants the migration scoped by next quarter.

The engineer assigned to scope it does something her predecessor could not. She opens a specification, not a codebase. The reconciliation rule that project recovered from decades-old batch code sits on her screen in the form it was left in: actor, business rule, postcondition, updated twice since through ordinary change requests, never rewritten from scratch. She does not page through Java source hunting for where a mismatch gets flagged. The rule is already written down, dated, and versioned. What took a recovery project the first time takes an afternoon the second time.

The engagement reported in Appendix A ends at the Java cutover. This second migration, and the afternoon it takes to scope, are this book's extrapolation from there, not a reported outcome.

■ CHAPTER PROMISE

By the end of this chapter, you can name what your organization's software economics actually reward once a specification outlasts the implementations built from it, and identify where your own team's business knowledge currently lives only in code nobody plans to reopen.

Reading time: 12 minutes

What the industry has spent decades protecting

For most of computing history, source code has been the asset organizations protect. Non-disclosure agreements guard it. Source-code escrow agreements insure against a vendor's collapse. Acquisitions price a company partly on the size and condition of its codebase. All of that spending rests on an assumption nobody examines closely: that the code is what makes the business hard to copy.

Run the thought experiment anyway. If a competitor obtained your entire technology stack overnight, every repository, every deployment script, would they be running your business tomorrow? Almost certainly not. They would have syntax. They would not have the decisions the syntax encodes: which exceptions to a rule the business actually tolerates, versus which ones only look tolerated because nobody ever enforced them; why a price calculation rounds the way it does; which regulatory workaround from a decade-old ruling is still load-bearing. None of that survives a copy-paste. Chapter 9 already showed why: the meaning of `ACCT-STATUS-CD = '07'` survived only in one operations lead's memory, and the deposit-ordering rule beside it had no agreed meaning at all.

The consequence is a mismatch. The organization spends its protective effort, legal, security, contractual, on an asset that would not reproduce the business by itself. Meanwhile the asset that would, the accumulated decisions behind the code, sits wherever Chapter 9 found it: in a departing employee's memory, in a batch job nobody has touched since the early 2000s, in no document at all.

The asset computing keeps moving

Software has organized itself around a different scarce resource in each era. In the mainframe decades, the asset was hardware: owning a machine capable of running a payroll job was the competitive edge. Once machines became common, the asset moved to software itself, applications expensive enough to write that owning the source code mattered. Then it moved again, to platforms: an operating system, a cloud provider, a marketplace, the layer everyone else's software had to run on top of. Each shift left the one before it in place. Hardware still matters. Software still matters. But the money and the advantage concentrated somewhere new each time.

This book's term for where that concentration is heading next is the **specification economy**: an arrangement in which the specification, not the code generated from it, is the asset an organization protects, versions, and reuses, because the specification is what survives every rewrite. That name is a proposal this book is making, not a category the industry has already settled on. What supports it is the pattern this book kept running into, most visibly in the bank's ledger, where the rule recovered once has already outlasted the code it was recovered from.

> **Code executes a decision. It does not explain the decision it is executing, and whoever holds only the code has to reconstruct the one thing that was never written down in it.**

Finding out which asset your organization actually protects

The question this chapter is asking, restated at the level of a single team, is checkable in an afternoon.

1. List the systems where replacing the technology keeps turning into a multi-year project instead of a straightforward rewrite. Not every brownfield system qualifies, only the ones where the cost is understanding rather than translation.
2. For each one, ask what would actually be lost if the code disappeared: the syntax, or the understanding behind it. If a departing team could hand the next team a specification instead of a repository, and the next team could implement it correctly, the organization's real asset is that specification, whether anyone has called it one yet.
3. Compare that list against the systems your organization already pays to protect through security review, access control, escrow, and backup policy. A system on the first list and not the second is where the protection spend is pointed at the wrong asset.

One rule, three implementations

The bank's ledger is this chapter's case for a reason. It is the one running example in this book old enough to have already outlived a technology generation, and specific enough to follow one rule at a time. The rule below came out of Chapter 9's recovery project, recovered from the ledger's nightly batch job alongside the withdrawal rules that chapter walks through. Nothing about its wording depends on the language that ends up executing it.

> **Reconcile Daily Ledger: recovered business rule**
>
> `Actor: nightly reconciliation job, unattended.`
>
> `Business rule: an account's ledger balance and its transaction history must sum to the same value at close of business. A mismatch of any amount holds the account for manual review before the next posting cycle.`
>
> `Postcondition: every open account either reconciles automatically or is flagged with the specific transaction range under review, never left unreconciled overnight.`

THREE IMPLEMENTATIONS, ONE RULE

- **Late 1980s, COBOL:** the rule as originally written, recoverable only by reading the batch job line by line. No other record of it survived.
- **After the recovery project, Java:** the rule as that project captured it, the same mismatch-and-hold behavior, now stated as a specification instead of buried in a nightly batch script.
- **Next migration, platform undecided:** the technology under review changes; the rest of the specification does not. The team scoping the move starts from the rule above, not from Java source, and the rule stays testable regardless of what ends up running it.

Riverside Clinics never faced this test; its booking specification is only a few chapters old by comparison. The bank's ledger has, and so far the specification has held where the COBOL it was recovered from did not.

What stays speculative, and what a specification cannot yet do

▲ **WARNING**

Two ideas this chapter gestures toward are not yet working practice anywhere in this book's case material. The first is regenerable software: swap the implementation skill, regenerate the system on a new stack, the rest of the specification unchanged. The second is specifications treated as protected intellectual property the way source code is now, with the legal and tooling support that implies. Both are plausible directions, consistent with everything else in this chapter. Neither is a documented practice yet, and this book will not claim otherwise.

This book uses the phrase **knowledge debt** for the pattern behind most of Chapter 9's recovery work: undocumented business rules, inconsistent terminology across systems, requirements nobody reconciled, assumptions a team stopped questioning. Knowledge debt is not an established industry term the way technical debt is. It's offered here as a name for something this book's case material kept describing without naming.

Treating "we wrote a specification" as the finish line	A badly written specification is worth as little as undocumented code. It just fails later, when someone tries to implement from it and can't.
Assuming legal protection for specifications works like it does for code	Copyright and trade-secret practice were built around source code. Nobody in this book's case material has a working answer yet for what protects a specification the same way.

Field guide: locating knowledge that lives only in code

- ☐ Could a new hire explain why this rule exists, or only what the code currently does?
- ☐ If the person who last touched this system left tomorrow, could someone else recover its reasoning within a week, or would it take a project like Chapter 9's?
- ☐ Do two systems in your organization implement the same business rule differently, because nobody wrote down which version is correct?
- ☐ Is there a rule your team enforces in code that no business stakeholder could check against a written description of it?

Three or more answers pointing to code as the only record mark a system where your organization's knowledge debt, proposed term or not, is concentrated.

What to remember

- Every prior shift in computing, hardware, then software, then platforms, moved what an organization treats as the asset worth protecting. Nothing here proves specifications are the next move; the bank's ledger is one case where the pattern has held so far.
- A stolen codebase does not reproduce a business, because what makes a business hard to copy survives as decisions, not as syntax a thief could read off a stolen repository.
- The bank's ledger specification recovered in Chapter 9 has already outlived one migration. This chapter's field guide is how to find the systems in your own organization where that hasn't happened yet.
- Regenerable software and specifications protected the way code currently is remain speculative. Treat them as a direction worth watching, not a workflow to adopt this quarter.
- This book's bet is that competitive advantage moves toward whoever understands their own business most precisely and has written it down, not toward whoever ships code fastest.

QUESTIONS FOR YOUR TEAM

- Which of your systems, handed to a brand-new team tomorrow with nothing but the source code, would take that team the longest to understand rather than to rebuild?
- Where does your organization currently spend the most effort protecting code that, by itself, would not let a competitor reproduce your business?

ONE ACTION TO TAKE NEXT

Pick one system from this chapter's field guide. Write, on a single page, the one business rule inside it that nobody could explain to a new hire without opening the code. That page is the first entry in a specification, whether or not it looks like one yet.

TRANSITION. The Afterword stops arguing and takes stock: what this book actually claimed, what it left out on purpose, and what would have to be true for its central bet to fail.

The specification economy ends here.